

HACKATHON PROBLEM STATEMENT REPORT

Production-Grade B2B Procurement & Automated Credit Management System

Project Code Name:	SmartB2B
Target Architecture:	MERN Stack (MongoDB, Express.js, React.js, Node.js) + Event-Driven Microservices Layer
Complexity Level:	Advanced / Industry-Standard (Enterprise SaaS)
Document Version:	1.0.0 (Hackathon Reference Guide)

1. Context & Industry Background

In emerging economies, the unorganized retail sector—comprising millions of local kirana stores, independent pharmacies, and neighborhood hardware merchants—forms the backbone of the retail commerce ecosystem. Despite high digital adoption at the consumer level (via instant payment UPI networks), the Business-to-Business (B2B) procurement workflow between local retailers and wholesale distributors remains deeply fragmented, archaic, and analog.

Currently, retailers place bulk orders using unstructured communication channels such as WhatsApp text messages, phone calls, or physical interactions with traveling sales representatives. Wholesalers, handling thousands of line items daily, manually parse these inputs, resulting in severe inefficiencies, inventory discrepancies, and delivery delays.

2. The Core Problem Statement

The fundamental friction in the unorganized B2B supply chain lies in the breakdown of communication, transactional data validation, and automated financial accounting. Specifically, three critical structural problems exist:

- **Information Asymmetry & Inventory Drift:** Retailers lack real-time visibility into a distributor's available stock. Orders are routinely accepted for items that are out of stock, leading to delivery fulfillment cuts and manual recalculations.
- **Fragmented Order Capture:** Managing thousands of incoming multi-line orders via WhatsApp leads to high operational overhead and order leakage for distributors. There is no centralized system to queue, prioritize, and track order fulfillment statuses.
- **The "Udhaar" (Credit/Ledger) Tracking Crisis:** Over 70% of local B2B transactions rely on short-term business credit. Bookkeeping is manually updated in physical registers (khata books) or standalone accounting software. This creates asynchronous ledgers, balance mismatch disputes, and bad debt exposure due to the lack of hard credit limit enforcement at the exact moment of checkout.

The Hackathon Challenge

Design and build a scalable, secure, and resilient multi-tenant SaaS platform that digitizes the end-to-end B2B order lifecycle, automates credit limit verification through strict database transactions, provides real-time fulfillment pipelines, and eliminates manual billing reconciliations.

3. Functional Requirements & Technical Specifications

The platform must support two discrete front-end applications or portal interfaces integrated via a highly unified, secure backend service tier.

3.1. Role-Based Access Control (RBAC) & Tenant Isolation

- **Wholesaler (Distributor) Panel:** Inventory management, bulk pricing tier configurations, retailer credit limit onboarding, live fulfillment pipeline dashboard, and ledger analytics.
- **Retailer Mobile/Web App:** Digital product catalog browsing, smart cart with credit check verification, historical order tracking, real-time dispatch alerts, and live digital credit statement.

3.2. Real-Time Distributed Order Processing

The ordering pipeline must be event-driven. When a retailer checks out, the transaction must immediately alert the wholesaler's dashboard without requesting page updates. If two retailers attempt to procure overlapping inventory that exceeds physical stock limits, the backend must elegantly isolate the requests, apply strict locking/concurrency control, and gracefully fail the second checkout without corrupting state data.

3.3. Automated Digital Ledger & Acid-Compliant Credit Engine

Every retailer profile holds an allocated `credit_limit` and a dynamic `outstanding_balance`. The core business rule is absolute: *New orders are blocked immediately if the current order value plus the outstanding balance surpasses the credit limit.* Because this involves concurrent checks on both financial ledgers and product catalogs, the checkout must run inside a strict database transaction block.

4. Technical Stack & Implementation Architecture

To mirror genuine industry engineering patterns, the solution must look beyond simple CRUD design patterns and utilize specialized tools for background queues, fast lookups, and transactional data integrity.

Layer / Component	Technology Stack	Industrial Engineering Purpose
Frontend Interface	React.js, Tailwind CSS, Redux Toolkit	Optimized state management for fast catalog search, infinite scrolling, and cross-tab reactive states.
Backend Application	Node.js (Express.js or NestJS)	Asynchronous, non-blocking asynchronous event loop handling highly concurrent connection pools.
Primary Database	MongoDB (Replica Set configuration)	Document model for flexible catalog structures, combined with multi-document ACID transactions for financial ledger integrity.
Real-Time Transport	Socket.io / WebSockets	Full-duplex channel enabling instantaneous order notifications and live status updates on dispatch.
In-Memory Cache	Redis	Caching high-read product catalogs and session storage to bypass primary database stress.
Asynchronous Workers	BullMQ / Redis Streams	Offloading heavy background computations like dynamic PDF generation and transactional email dispatches.
Object Storage	AWS S3 (or MinIO equivalent)	Secure, immutable remote cloud hosting for auto-generated business invoices and compliance documents.

5. Suggested Database Schema Blueprint

To prevent structural failures, entities must be linked via reference patterns while ensuring transactional stability across high-volume write boundaries. Below is a structured architectural guide for the critical collections:

Users & Roles Collection

```
{
  "_id": "ObjectId",
  "name": "Mahavir Distributors",
  "email": "contact@mahavir.com",
  "role": "WHOLESALER",
  "isActive": true,
  "createdAt": "ISODate"
}
```

Products (Inventory) Collection

```
{
  "_id": "ObjectId",
  "wholesalerId": "ObjectId",
  "sku": "SKU-MED-10293",
  "name": "Paracetamol 650mg Bulk Pack",
  "category": "Pharmaceuticals",
  "stockQuantity": 1450,
  "unitPrice": 120.00,
  "reservedStock": 50
}
```

Retailer Credit Profile Collection

```
{
  "_id": "ObjectId",
  "retailerId": "ObjectId",
  "wholesalerId": "ObjectId",
  "creditLimit": 50000.00,
  "outstandingBalance": 12500.00,
  "status": "ACTIVE"
}
```

Ledger Transactions Collection

```
{
  "_id": "ObjectId",
  "retailerId": "ObjectId",
  "wholesalerId": "ObjectId",
  "orderId": "ObjectId(Nullable)",
  "type": "DEBIT", // DEBIT increases outstanding, CREDIT (payment) decreases it
  "amount": 4200.00,
  "runningBalance": 16700.00,
  "description": "Order Placement #ORD-99211",
  "timestamp": "ISODate"
}
```

6. Advanced Engineering Challenges (Interview Standout Points)

To score maximum technical points, candidates must design explicitly for the following distributed systems edge cases:

1. **Preventing Double Spend on Credit:** If a retailer opens two browser tabs and submits two separate orders simultaneously, a naive backend would check the balance twice before updating, accidentally allowing the retailer to exceed their credit limit. The code must implement MongoDB Session Transactions (`session.startTransaction()`) with an `isolated` read/write lock level.
2. **Managing Stock Race Conditions:** High-concurrency checkouts can cause stock numbers to drop below zero (Over-selling). Implement atomicity at the database driver level using atomic updates like `$inc: { stockQuantity: -orderedQty }` combined with an explicit query filter ensuring `stockQuantity: { $gte: orderedQty }`.
3. **Resilient Background Processing:** Generating a multi-page PDF invoice and pushing it to AWS S3 takes anywhere from 500ms to 2 seconds. Doing this directly inside the HTTP request cycle blocks the Node.js event loop and delays the user's response. Offload this work completely to an asynchronous BullMQ queue worker pattern.